RUNCORE: Energy-Aware Resource Allocation for Hybrid CPU Warehouse Systems

***Abstract*—Asymmetric multicore processors (AMPs) combine high-performance and energy-efficient cores under a shared instruction set architecture to improve efficiency, and have been increasingly adopted in warehouses and big data centers. At the same time, AMPs are challenging for conventional container mechanisms found in these huge systems, since they usually assume the use of equal and interchangeable cores. Therefore, in this paper, we propose RUNCORE, an energy-aware adaptive resource manager for containerized workloads on AMP systems. Using profiling, RUNCORE first adapts the thread level parallelism of each application (i.e., reduces the number of concurrent threads when necessary), then ranks feasible core allocations by Energy-Delay Product (EDP) and applies a cooperative three-phase strategy: *(1)* priority-driven best-fit allocation, *(2)* cooperative downgrade, and *(3)* marginal-gain redistribution of residual cores. We evaluate RUNCORE with Dockerized kernels from the NAS Parallel Benchmarks Suite on an Intel Core i9-12900K (Alder Lake) under a high-contention campaign with 7 candidate configurations per workload and 50 concurrent containers. Normalized to the default serial baseline, RUNCORE reduced EDP by 91.2% on average, while also improving the mean EDP over classical container heuristics by 56.51% and a heterogeneity-blind variant by 49.92%.**

I. Introduction

The growing adoption of asymmetric multicore processors (AMPs) is reshaping modern computing. big.LITTLE-style architectures that combine high-performance and energy-efficient cores [1], [2], including Intel Alder Lake/Meteor Lake and Apple M1/M2, are increasingly used in client, edge, cloud, and development settings [3], [4]. Their goal is to balance performance and energy efficiency by matching workloads to heterogeneous cores.

At the same time, container-based execution (e.g., Docker, Podman, Apptainer) has become the *de facto* model for cloud and, increasingly, warehouses and High-Performance Computing (HPC) deployments. Containers provide lightweight isolation, fast provisioning, and near-native performance, making them attractive for parallel and distributed workloads [5], [6], [7]. As a result, many applications run parallel code (often OpenMP-based) inside containers and rely on runtimes and operating system (OS) schedulers for transparent resource management. Commercial cloud infrastructures have already begun exposing AMP-based systems to users: AWS provides Apple M-series processors through EC2 Mac instances (e.g., `mac2-m1ultra.metal`, `mac2-m2pro.metal`) [8], demonstrating the practical viability of hybrid CPU architectures in production cloud environments. However, these platforms primarily target dedicated single-tenant development workflows and rely heavily on vertically integrated hardware–software optimization.

A key reason is that system virtualization introduces extra challenges for AMPs: it hides physical heterogeneity from guest OSes and container runtimes, which may then schedule threads as if all cores were identical [9]. However, on AMPs, the same parallel workload can show very different performance, power, and energy behavior across core types. Compute-bound applications usually benefit from P-cores (higher instructions per cycle/frequency), while memory-bound or synchronization-heavy applications may achieve similar throughput on E-cores with much lower power. Therefore, blindly maximizing thread parallelism on P-cores or uniformly distributing threads across all available cores can lead to sub-optimal performance-energy trade-offs, increased contention for shared resources, and inefficient utilization of the cores.

The issue becomes even more pronounced when multiple containers co-execute on the same AMP platform, as containers compete for scarce high-performance P-cores while E-cores may remain underutilized due to the lack of heterogeneity-aware scheduling policies. Existing vertical and horizontal autoscaling mechanisms, which dynamically adjust the computational resources assigned to applications, typically rely on coarse-grained metrics such as aggregate CPU and memory utilization [10], [11], and therefore fail to capture the complex interactions among thread-level parallelism (TLP), heterogeneous core asymmetry, and concurrent workload execution. Furthermore, currently available commercial AMP cloud offerings, such as AWS EC2 Mac instances [8], primarily rely on host-level scaling approaches in which additional physical machines must be provisioned through dedicated host allocations with fixed reservation periods (e.g., 24-hour minimum allocation windows). This model limits scheduling flexibility, and consequently, a substantial portion of the performance-energy optimization potential of AMP architectures remains unexplored, motivating the development of heterogeneity-aware scheduling techniques capable of coordinating resource-management decisions across multiple system layers.

In this work, we address this challenge by proposing **RUNCORE**, a framework for energy-efficient placement of containerized applications in cloud/HPC environments equipped with AMPs. Our approach leverages information about the applications' TLP to determine: ***(i)*** the right level of parallelism of each application, by decreasing the number of concurrent threads considering the current load and status of the system; and ***(ii)*** an appropriate mapping of threads and containers to heterogeneous core types prior to execution. By guiding runtime placement through offline analysis, RUNCORE en-

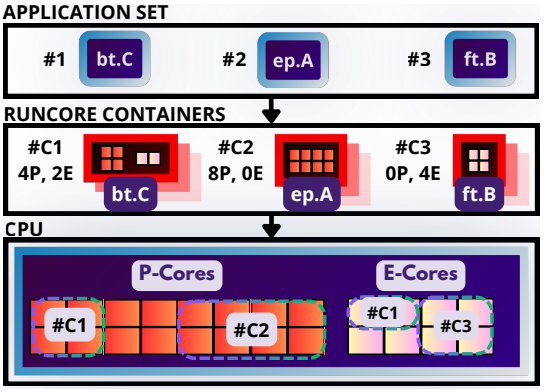


Fig. 1: Overview of the RUNCORE resource management process.

ables efficient concurrent execution of multiple containers, reducing makespan while improving the Energy-Delay Product (EDP). Figure 1 presents an overview of the RUNCORE resource management process. Initially, the user gives a batch of applications (**APPLICATION SET**) to run (for example, bt.C, ep.A, and ft.B). RUNCORE then orchestrates the creation and deployment of containers (**RUNCORE CONTAINERS**), assigning each application to its own container (bt.C $\rightarrow$ #C1, ep.A $\rightarrow$ #C2, ft.B $\rightarrow$ #C3), mapping a given thread count configuration (#P, #E), one-to-one for Performance (P) and Energy-Efficient (E) cores. The resource manager transparently allocates one or more containers to a processor, ensuring that each container will be given a set of cores composed of any combination of E- and P- cores (represented by **CPU** in Fig. 1).

Our experimental evaluation compares RUNCORE against AMP-unaware solutions across 10 batches, each comprising a different application set. For each batch, we evaluate 7 candidate configurations of E-core and P-core thread counts and run 50 concurrent containers. We also demonstrate that RUNCORE outperforms state-of-the-art solutions, including homogeneous, heuristic, and pinned-serial approaches, and also achieving mean EDP reductions of 91.2% compared to the default serial baseline, without sacrificing neither energy or execution time. Our results highlight the importance of heterogeneity-aware planning and global coordination, particularly in oversubscribed environments.

The remainder of this paper is organized as follows. Section II discusses the basic concepts and related work on AMPs and virtualization. Section III provides an overview of the proposed approach. Section IV outlines the experimental methodology, while Section V presents the evaluation results. Finally, Section VI concludes the paper.

II. Background and Related Work

A. Asymmetric Multicore Processors (AMP)

Asymmetric multicore processors (AMPs) improve energy efficiency by integrating cores with different performance and power profiles under a shared instruction set architecture [12]. Initially widespread in mobile systems (ARM DynamIQ/big.LITTLE) [12], AMP designs now extend to desktop and server platforms, including Apple M2 and Intel Alder Lake [13]. These architectures combine high-performance P-cores and energy-efficient E-cores: P-cores deliver higher performance at higher power, whereas E-cores provide lower performance at lower power [12]. While this heterogeneity enables performance-energy optimization, it also complicates runtime core allocation.

Recent research on AMPs has mainly focused on two directions: ***(i)*** microarchitectural mechanisms and characterization [14], and ***(ii)*** OS-level scheduling strategies [15]. In general, these efforts improve performance-energy trade-offs by leveraging heterogeneity inside a single machine, but they typically do not address container-level multi-tenancy, nor do they provide an application-centric resource manager that jointly decides both the degree of parallelism (TLP) and where those threads run (P-cores vs. E-cores) under shared-resource interference. Given that, Shimchenko et al. [16] investigated energy-efficient scheduling of garbage collection tasks, demonstrating that selective core assignment can reduce energy consumption while satisfying latency constraints. Saez et al. [12] conducted a comparative analysis of Intel's Thread Director for Alder Lake processors against hardware performance-counter–based asymmetric schedulers, evaluating their effectiveness in heterogeneous environments. Schöne et al. [17] characterized the performance and energy behavior of an Alder Lake system, analyzing the influence of power-management mechanisms, including dynamic frequency scaling and core idle-state transitions.

B. Parallel Computing in Virtualized and Cloud Environments

Parallel applications exhibit diverse behaviors and resource demands. Some are compute-intensive and benefit from high instruction throughput, while others are memory-bound or dominated by synchronization. Although AMPs can improve both performance and energy efficiency, they also introduce extra execution and scalability challenges beyond those of homogeneous multicores. Key challenges include:

- *Imbalanced execution across core types:* Performance asymmetry between P-cores and E-cores can worsen load imbalance, especially for inherently uneven workloads [12]. Small per-thread timing differences can propagate into large barrier delays.
- *Saturation of functional units:* Workloads with high instruction-level parallelism (ILP) can achieve strong performance with relatively few threads [18]. Because AMPs usually provide fewer P-cores (better suited to exploit ILP), oversubscribing compute-intensive threads on E-cores can cap throughput.
- *Off-chip bus congestion:* Concurrent memory accesses (e.g., DRAM) can saturate bandwidth, increasing contention and latency, particularly for memory-bound workloads [19]. On AMPs, heterogeneous access patterns may further amplify this effect.
- *Synchronization overhead:* Frequent synchronization increases waiting time and reduces parallel efficiency [19].

Core asymmetry intensifies this effect when faster P-cores wait for slower E-cores.

- *Intensive shared memory access:* Repeated access to shared structures can increase coherence traffic and contention, creating bottlenecks that limit scalability and throughput [18]. Cache-capacity and latency differences across core types can intensify these bottlenecks.

These difficulties restrain the availability of AMP cloud instances in multi-tenant scenarios; as mentioned in Section I, AWS EC2 Mac instances (`mac2.metal`, `mac2-m1ultra.metal`, `mac2-m2.metal`, `mac2-m2pro.metal`) operate in single-tenant bare-metal mode, which simplifies management and reduces shared-resource interference.

A number of studies have investigated the performance and behavior of parallel applications deployed in cloud infrastructures [20], [21], [22]. However, these solutions generally rely on static configurations and do not incorporate adaptive mechanisms. In contrast, several studies concentrate on optimizing standalone executions using techniques such as thread tuning (TT) [23], [24]. These approaches typically assume exclusive access to computational resources, modeling performance as a function of thread count under stable hardware conditions. As a result, they disregard resource contention effects that arise in multi-tenant environments, such as shared cache interference, memory bandwidth competition, and core-level oversubscription. To the best of our knowledge, only the work of Silva et al. [25] explicitly considers multitenancy in the context of TT by coordinating tuning decisions across co-located applications. Yet, their approach assumes homogeneous multicore architectures, and therefore does not address how tuning interacts with core-type asymmetry.

Finally, only a limited number of studies explicitly address the interaction between AMPs and virtualization or cloud scheduling. Kwon et al. [26] present one of the earliest works on this topic, proposing a hypervisor mechanism that transparently maps virtual machines (VMs) to heterogeneous cores without requiring modifications to guest OSes or applications. Later, Teabe et al. [27] incorporate asymmetry awareness into cloud VM schedulers by weighting CPU allocations according to core capabilities and adapting decisions based on observed execution rates. While advancing AMP-aware virtualization, this line remains centered on VM-level allocation rather than a container-level runtime that jointly manages ***(i)*** *how many threads each application should use* (TLP), ***(ii)*** *which core types those threads should occupy* (P/E asymmetry), and ***(iii)*** *how multiple containers should be coordinated* under shared-resource contention.

C. Our Contribution

Compared with prior AMP-aware solutions that focus mainly on hardware/kernel mechanisms [12], [17], VM-level asymmetry management [26], [27], or homogeneous thread-tuning assumptions [25], RUNCORE targets the missing integration point: coordinated, container-level resource management on heterogeneous cores. Concretely, RUNCORE closes three gaps simultaneously:

- **Thread-level parallelism (TLP):** instead of assuming a fixed thread count, RUNCORE explicitly selects and revises each container's effective parallelism level (e.g., scaling threads up/down) to avoid inefficient oversubscription and synchronization/bandwidth bottlenecks.
- **Core-type asymmetry:** RUNCORE assigns work considering heterogeneous performance/power profiles (P-cores vs. E-cores), preventing systematic slowdowns due to imbalance and avoiding placing throughput-critical threads on unsuitable cores.
- **Multi-container contention:** RUNCORE makes decisions in a coordinated manner across co-located containers, accounting for shared-resource interference rather than optimizing each application in isolation.

By integrating these three control dimensions into a single runtime decision pipeline, RUNCORE provides an adaptive mechanism that is not achieved by existing approaches that treat TLP control, asymmetry handling, and multi-tenant coordination as separate problems.

III. RUNCORE Resource Manager

RUNCORE is an energy-aware resource manager for containerized workloads on cloud/HPC AMP systems. It orchestrates the concurrent execution of a batch of K containers on a platform composed of C_P Performance-cores (P-core) and C_E Efficiency-cores (E-core), with the objective of minimizing the aggregate Energy-Delay Product (EDP) of the workload batch.

Our proposal operates in two stages: **offline** and **online**. In the **Offline Stage**, the RUNCORE profiles application executions across multiple P-core/E-core configurations, generating a ranked LUT. In the **Online Stage**, it serves as a runtime, three-phase resource manager that monitors the batch specification and the current infrastructure state (e.g., available cores and running containers) to determine core allocations, which are enforced through container runtime mechanisms. The following sections detail the design and operation of the proposed resource manager.

A. *Offline Stage: Profiling and EDP Ranking*

In this stage, each candidate workload is profiled offline across a set of (p, e) core configurations, where p denotes the number of P-core threads and e the number of E-core threads. For each configuration, the profiling stage measures three metrics: total energy consumption E (joules)[1], wall-clock execution time T (seconds), and the resulting EDP $= E \times T$ (see section IV).

The profiling results are then filtered (see Figure 2), mapping each workload w from the set of applications in the batch W and configuration (p, e) to the triple $(E_{w,p,e},\ T_{w,p,e},\ EDP_{w,p,e})$, and are sorted by increasing EDP, yielding a **ranked LUT** of size $N \times |W|$, where each

[1]In this section, *E* (*italicized*) represents the measured energy consumption, whereas E (or E-) throughout the paper refers to the E-core type.

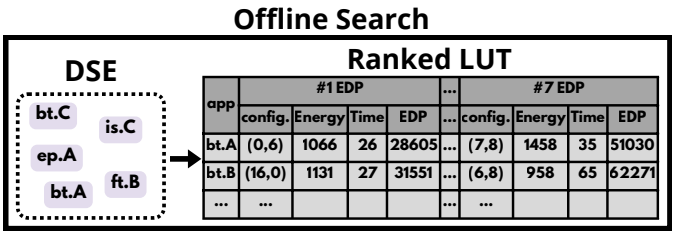


Fig. 2: *Offline Stage*: Creation of the EDP lookup table (LUT) and configuration ranking.

element in N is a tuple (w, E, T, EDP). At runtime, the resource manager uses the top-N entries for each w in this ranking, resulting in a ranked LUT that contains significantly fewer configurations than the complete DSE (5.51% in our experiments; see Section V). Larger values of N give the resource manager more flexibility to accommodate resource constraints, while smaller values restrict it to the most EDP-efficient configurations.

B. ***Online Stage:*** *Three-Phase Pipeline*

Given a batch of K workloads (let's assume that $K = |W|$) and their ranked LUT, the resource manager assigns cores through a three-phase pipeline: *(1)* priority-driven best-fit allocation, *(2)* cooperative downgrade, and *(3)* marginal-gain redistribution. This pipeline is executed iteratively until all application in W has been deployed. Figures 3a, 3b, and 3c illustrate the current stage, providing an example of an iteration of our pipeline, detailed next.

Phase 1: Priority-Driven Best-Fit Allocation. Workloads are first sorted from the batch in descending order of a **composite priority score** defined as

$$priority(w) \;=\; T_w^{\star} \;\times\; \sigma(w), \tag{1}$$

where $T_w^{\star}$ is the execution time of workload w at its best-ranked configuration (LUT rank 1), and $\sigma(w)$ is its **EDP sensitivity**:

$$\sigma(w) \;=\; \frac{EDP_w^{(r)}}{EDP_w^{(1)}}, \qquad r = \min(5,\, N), \tag{2}$$

i.e., the ratio between the EDP at LUT rank r and at LUT rank 1. Workloads that are both long-running and highly sensitive to configuration quality receive the highest priority and are scheduled first, ensuring they obtain their preferred core allocations before resources are consumed by less critical workloads.

For each workload (in priority order), the resource manager walks the LUT and selects the first configuration whose EDP is within an **adaptive tolerance factor** $\alpha(w)$ of the workload's best configuration and whose core requirements fit within the currently available P-core and E-core pools. A configuration at rank i is *acceptable* if

$$EDP_w^{(i)} \;\le\; \alpha(w) \;\times\; EDP_w^{(1)}. \tag{3}$$

The tolerance factor adapts to the workload's execution time via linear interpolation:

$$\alpha(w) = \begin{cases} \alpha_{\max} & \text{if } T_w^{\star} \le T_{\text{short}}, \\ \alpha_{\max} + \frac{T_w^{\star} - T_{\text{short}}}{T_{\text{long}} - T_{\text{short}}} \cdot (1 - \alpha_{\max}) & \text{if } T_{\text{short}} < T_w^{\star} < T_{\text{long}}, \\ 1 & \text{if } T_w^{\star} \ge T_{\text{long}}, \end{cases} \tag{4}$$

where $\alpha_{\max} = 1.18$, $T_{\text{short}} = 5\,\text{s}$, and $T_{\text{long}} = 102\,\text{s}$, thereby constraining the adaptive tolerance factor to the interval $1 \le \alpha(w) \le \alpha_{\max}$. The constants are derived empirically from the workload profiling data obtained in our experiments: $\alpha_{\max}$ corresponds to the $mean + \sigma$ ratio of rank-5 EDP to rank-1 EDP across all workloads; T_{short} and T_{long} correspond to the median and maximum best-configuration execution times, respectively.

Short-running workloads thus accept configurations up to 18% (1.18) worse than their optimum (since their absolute EDP contribution is small), while long-running workloads are constrained to their best available configuration. In Figure 3a, bt.C is the longest application, which is then selected for allocation first. If no acceptable configuration fits, the workload is marked as *deferred* and is not reconsidered until the next iteration.

Phase 2: Cooperative Downgrade. For each deferred workload d, the resource manager evaluates whether downgrading an already-allocated workload (the *donor*) to a cheaper (fewer core) configuration would free enough cores to schedule d. The key observation is that both the donor and the deferred workload must eventually execute; the question is whether running them *concurrently* (with a worse donor configuration) yields a lower combined EDP than running them *sequentially* (each at its preferred configuration). In Figure 3b, ep.A is downgraded, for the heuristic sees that the net gain from allocating is.C is positive ($\Delta > 0$).

Following this example, let the donor (e.g. ep.A in Figure 3b) currently use configuration c and consider downgrading it to configuration c' (with $p_{c'} \le p_c$ and $e_{c'} \le e_c$), enabling deferred workload d (e.g. is.C in Figure 3b) to use configuration c_d. It compares:

$$EDP_{\text{seq}} = E_{\text{donor}}^{(c)} \times T_{\text{donor}}^{(c)} + E_d^{(1)} \times T_d^{(1)}, \tag{5}$$

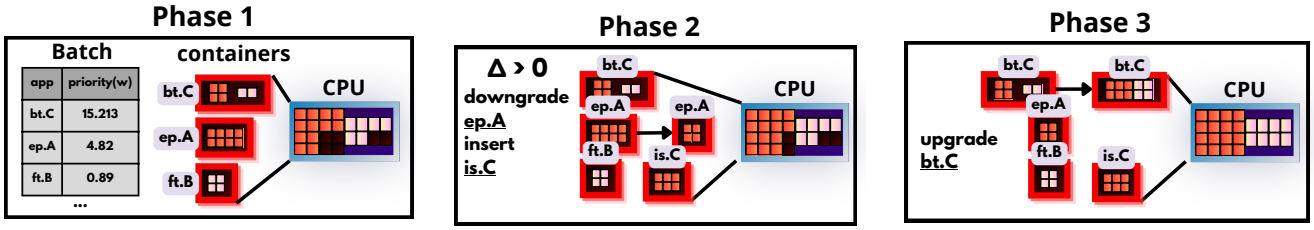
$$EDP_{\text{con}} = \big(E_{\text{donor}}^{(c')} + E_d^{(c_d)}\big) \times \max\big(T_{\text{donor}}^{(c')},\, T_d^{(c_d)}\big). \tag{6}$$

In the sequential case, the two workloads execute one after the other, so their individual EDPs simply add up. In the concurrent case, their energies are additive (both consume power simultaneously) but the wall-clock time is the maximum of the two overlapping execution times.

The **net gain** of the swap is:

$$\Delta = EDP_{\text{seq}} - EDP_{\text{con}}. \tag{7}$$

The swap is performed only when $\Delta > 0$. Among all valid (*donor*, c', d, c_d) combinations, it selects the one with the highest Δ. The process repeats until no further beneficial



(a) RUNCORE allocates bt.C, ep.A, and ft.B to the available cores.

(b) RUNCORE deallocates some cores for ep.A and gives them to is.C.

(c) RUNCORE allocates additional vacant cores to bt.C.

Fig. 3: *Online Stage*: Runtime resource management phases.

swaps exist.

Phase 3: Marginal-Gain Redistribution. After Phases 1 and 2, some P-core or E-core threads may remain unallocated due to configuration granularity. It reclaims these idle cores through an iterative **marginal-gain** optimization. Figure 3c shows this behavior when reclaiming 2 E-Cores and 2 P-Cores for bt.C, marginally improving the overall EDP. At each iteration, for every allocated workload that is not already at its rank-1 configuration, it evaluates all possible upgrades (i.e., configurations with a lower rank index that require additional cores) and computes the **EDP gain per extra core**:

$$g(w, i \to j) \;=\; \frac{EDP_w^{(i)} - EDP_w^{(j)}}{(p_j - p_i) + (e_j - e_i)}, \tag{8}$$

with $j < i$, $p_i \le p_j$, $e_i \le e_j$, where i is the current configuration index and j is the candidate upgrade. The workload–upgrade pair with the highest g receives the additional cores. The process continues until no upgrade is feasible within the remaining free cores.

This strategy directs idle resources to the workload with the highest EDP improvement per additional core, thereby maximizing resource utilization. Applications whose rank-1 configuration already requires few cores are naturally accommodated by Phases 1 and 2, since their modest resource demands make allocation at their optimal EDP point highly likely. Consequently, allocating additional cores beyond the profiled rank-1 configuration is neither necessary nor desirable, as it would reduce the availability of cores for other workloads and potentially lower overall system efficiency.

C. Transparency and Deployment

RUNCORE framework operates entirely at the container orchestration level, making it **transparent to applications**. In particular, it: ***(i)*** does not require modifications to application source code or binaries; ***(ii)*** does not require changes to container images, and ***(iii)*** does not require modifications to the guest OS.

All scheduling decisions are implemented externally through container runtime controls, specifically CPU-set affinity (`cpuset_cpus`) and environment variables (`OMP_NUM_THREADS`, `GOMP_CPU_AFFINITY`), which enables seamless deployment in existing containerized environments.

IV. Experimental Methodology

Environmental Setup. Our experiments were executed on a workstation equipped with an Intel Core i9-12900K (Alder Lake) processor running Ubuntu 22.04 LTS with Linux kernel 5.15. The processor comprises 24 logical cores organized as follows: 8 P-cores capable of executing up to 16 threads with Hyper-Threading enabled and 8 E-cores organized into two clusters of four cores. Each P-core has a private 1.25 MiB L2 cache, each E-core cluster shares a 2 MiB L2 cache, all cores have private 32 KiB L1 instruction and 32 KiB L1 data caches, and all share a 30 MiB L3 cache. The system has 32 GiB of DDR5 memory.

Workload Suite. We considered nine kernels from NAS Parallel Benchmarks (NPB) [28], version 3.3, compiled with GCC 11.4 and OpenMP, covering a wide spectrum of computational characteristics: *Compute-bound* – BT (Block Tridiagonal), SP (Scalar Pentadiagonal), LU (Lower-Upper symmetric Gauss-Seidel); *Memory-bound* – CG (Conjugate Gradient), MG (Multigrid), FT (Fourier Transform), IS (Integer Sort); *Embarrassingly parallel* – EP (Embarrassingly Parallel); and *Unstructured* – UA (Unstructured Adaptive). Each kernel was compiled at three problem sizes (classes A, B, and C), yielding a total of 27 distinct workloads whose execution times range from approximately 5 s to over 100 s on the target platform.

Evaluated Metric for Energy-Efficiency. We evaluate the *Energy-Delay Product (EDP)* [29], which captures the trade-off between energy consumption (E) and performance (execution time, T), defined as $\mathrm{EDP} = E \times T$. Consequently, a configuration that significantly reduces execution time but slightly increases energy consumption tends to produce a better EDP, as does a configuration that minimizes energy at the cost of minor performance degradation. We measured T as the wall-clock elapsed time (in seconds) from the launch of the first container to the completion of the last container in the batch. For energy E, we used `perf stat -e power/energy-pkg/`, which reads the Intel RAPL counter to measure the total energy consumed by the CPU package during the batch execution.

Offline EDP Profiling. Before running experiments, each workload was profiled across all possible thread counts and thread-to-core allocations on the target AMP system (see section III-A). A core allocation is represented by a tuple (p, e), indicating the number of P-cores and E-cores assigned

to the container. We enumerate all combinations with $p \in \{0, \ldots, 16\}$ (hyperthreading enabled) and $e \in \{0, \ldots, 8\}$, subject to $p+e > 0$, yielding 152 configurations per workload.

Evaluated Strategies. We evaluated the proposed RUNCORE (denoted as RCORE in the plots) and a set of baselines that isolate ***(i)*** the impact of heterogeneity-aware mapping, ***(ii)*** classic time-sharing heuristics under oversubscription, and ***(iii)*** simple serial affinity policies. All strategies executed each workload inside a Docker container and, when applicable, enforced CPU affinity using Docker's `--cpuset-cpus` option. For OpenMP, we set `OMP_NUM_THREADS` according to the intended allocation and exported `GOMP_CPU_AFFINITY` to match the container CPU mask. Finally, we exercised these strategies on a heavy contention scenario with N=7 candidate configurations per workload and 10 differently randomized batches of K=50 concurrent containers. We selected N empirically, whereas varying it from 3 to 8 changed the results only marginally, yielding a coefficient of variation of just (0.43%). Thus, we derived and compared our proposal (named as RCORE in the charts) to the following strategies:

- **RCORE (HOMOGEN).** A homogenized version of RCORE, covering the same principles as the State of The Art work in [25] which ignores performance asymmetry by replacing (p, e) allocations with a single thread-count configuration t. Each container was pinned to any t currently free logical CPUs (no P/E distinction) with `OMP_NUM_THREADS`$= t$. The offline profiling sweep for this strategy is analogous to RCORE, but considering only a single thread-count parameter $t \in \{1, \ldots, 24\}$ and pinning each container to any t logical CPUs.
- **RR (ROUND-ROBIN).** A round-robin baseline that launched one container per workload and pinned *all* containers to the same shared cpuset consisting of all available logical CPUs at the beginning of the run. Containers were initially paused and then executed in strict FIFO rotation by unpausing each container for a fixed quantum ($q = 5\,\text{s}$ by default) before pausing and re-queuing it if it had not yet completed.
- **SJF (SHORTEST-JOB-FIRST).** A shortest-job-first baseline that did not use the EDP ranked LUT and did not partition cores across containers. Instead, it co-located the entire batch by pinning every container to the same full cpuset (all available logical CPUs) and setting `OMP_NUM_THREADS` to the size of that shared mask.
- **BL (Default).** A naive serial baseline that launched workloads one-at-a-time without an explicit CPU mask (i.e., the container could execute on all logical CPUs) and used `OMP_NUM_THREADS`$= 24$.
- **BL (P), BL (E), and BL (ALT P/E) (serial with affinity pinning).** Three serial baselines that executed the batch sequentially while pinning each container to fixed core sets: **BL (P)** pinned each workload to the P-core logical CPUs $\{0, \ldots, 15\}$ with `OMP_NUM_THREADS`$=$ 16; **BL (E)** pinned each workload to the E-core CPUs $\{16, \ldots, 23\}$ with `OMP_NUM_THREADS`$= 8$; and **BL (ALT P/E)** alternated these two affinity masks across successive workloads (even index $\rightarrow$ P, odd index $\rightarrow$ E).

It is worth noting that all experiments were conducted in an in-house environment designed to emulate large-scale cloud/HPC infrastructures. Consequently, the reported results should be regarded as optimistic compared to executions on commercial AMP cloud platforms. In particular, AWS EC2 Mac instances [8] rely on dedicated-host provisioning with minimum 24-hour allocation windows, meaning that reproducing the full experimental campaign would require weeks or even months to complete.

V. RESULTS

A. Overall Results

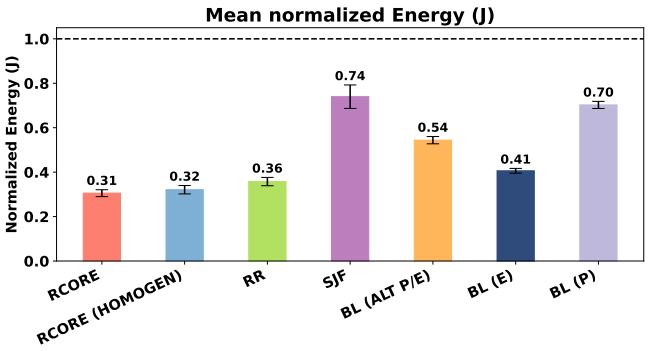
This section presents the overall results obtained by applying the RUNCORE's Online Stage. For that, Figures 4a, 4b and 4c summarize the mean normalized metrics (Energy, Execution Time, and EDP – y-axis) with standard error bars (Standard Error of the mean), considering all 10 different batches executed by the evaluated strategies (x-axis). These are normalized to the default serial baseline BL (values less than 1 means improvement).

EDP improvements. RCORE achieved the lowest EDP, with a mean normalized EDP of 0.088 (SEM $= 0.010$, 95% Confidence Interval - CI $[0.069, 0.107]$), corresponding to a 91.2% EDP reduction relative to BL, as shown in Fig. 4c. Among the other strategies, RR performed best (87.8% reduction), but still remained 27.86% worse than RCORE. This improvement is a consequence of cache temporal locality: by preserving core affinity (and thus reusing warm caches), RR can reduce cache warm-up penalties under heavy oversubscription, with 45.94% fewer last-level cache (LLC) load misses compared to RCORE. SJF exhibited substantially higher EDP (only 39% reduction relative to BL), indicating that selecting short jobs without heterogeneity-aware core assignment is insufficient under heavy oversubscription.

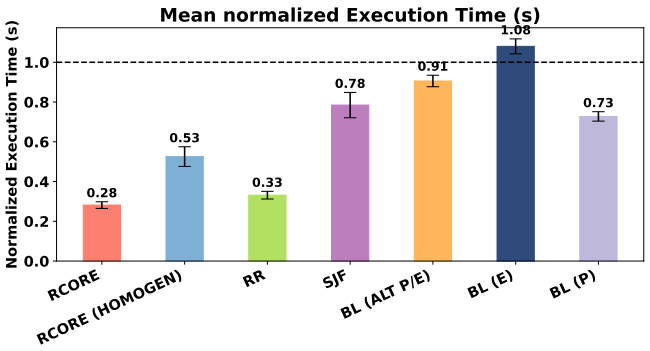
Energy and time decomposition. The EDP gains were driven by improvements in both dimensions (see Fig. 4a and Fig. 4b). RCORE reduced energy by 69.5% and execution time by 71.9%. RR achieved competitive yet still lower means (7.62% more energy and 6.95% longer execution time), confirming that even simple cyclic allocation benefits from inter-container parallelism, but it still falls short of the proposed RCORE.

Importance of asymmetry awareness. The RCORE (HOMOGEN) variant consumed a similar amount of energy (3.42% more) but executed significantly slower (46.66%) than RCORE, yielding a mean normalized EDP of 0.176. In other words, ignoring core heterogeneity roughly doubled EDP compared to the asymmetry-aware design, demonstrating that the correct mapping of workloads to P-/E-core resources is a prominent factor in the observed EDP improvements.

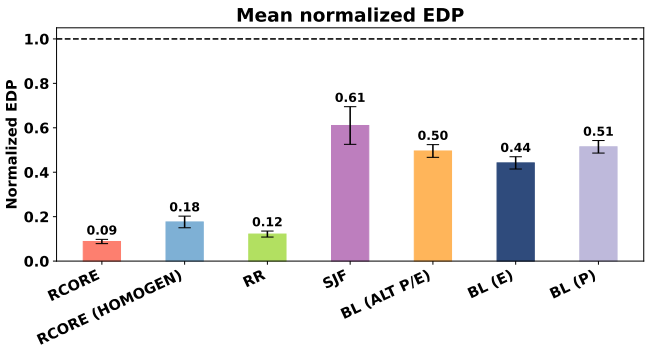
Serial baselines with core pinning. All pinned serial variants improved upon the naive BL reference, but remained far from RCORE. The best serial variant in terms of EDP was



(a) Normalized energy consumption.



(b) Normalized execution time.



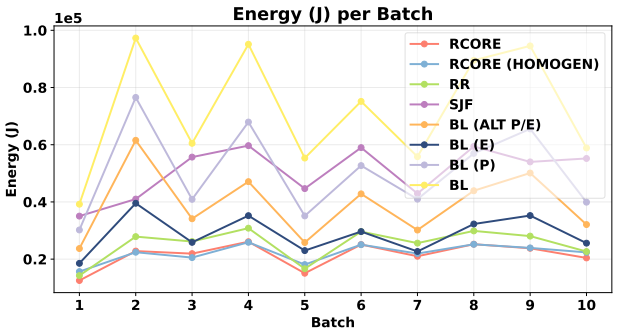
(c) Normalized EDP.

Fig. 4: Mean normalized metrics (baseline: BL, i.e., serial execution) with SEM error bars for N=7 and K=50: (a) energy consumption, (b) execution time, and (c) EDP.

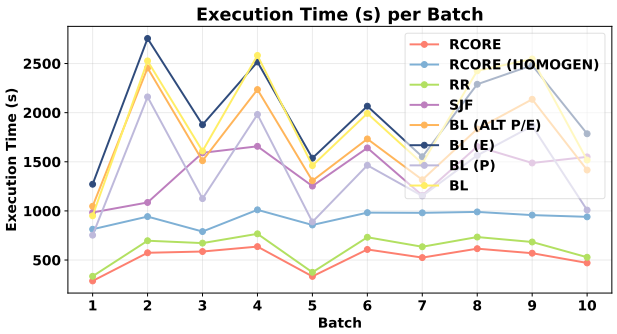
BL (E) with 57.8% reduction, reflecting lower energy (59.4% reduction) at the expense of longer runtime (8% increase). BL (P), however, despite reducing both energy and execution time, still incurred lower EDP gains than both BL (E) and BL (ALT P/E). Finally, these results indicate that simple affinity policies can partially mitigate energy waste, but cannot replace concurrency-aware planning when in oversubscribed environments.

B. Per-batch Analysis

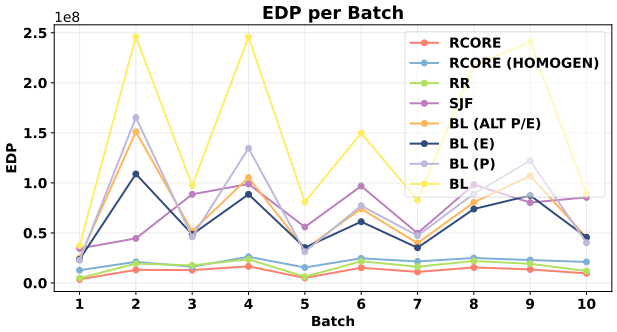
We now present the results considering per-batch execution. Figures 5a, 5b, and 5c provide a view similar to that of



(a) Per-batch energy.



(b) Per-batch execution time.



(c) Per-batch EDP.

Fig. 5: Per-batch metrics for N=7 and K=50: (a) energy consumption, (b) execution time, and (c) EDP.

the previous section, but now the analysis is broken down by application batch (the values on the x-axis correspond to the IDs of the evaluated batches). The charts present the raw values for all evaluated metrics.

The per-batch plots show that RCORE remained consistently better than all alternatives across the 10 batches. Quantitatively, its EDP gains ranged from 94.7% to 86.7% reduction, suggesting that our proposal's advantage persists across diverse randomly generated batches. It is worth noting as well that there are several batches (e.g., indexes 2, 4, 9) that are significantly worse across all BL variants in a uniform manner, shown in all plots for Energy, Execution Time, and EDP. These patterns are easily seen as a y-axis translation, which may suggest a proportional and fixed worsening of par-

allel scalability for some of the randomly selected applications for each batch.

C. Discussion

The results yield three key insights. ***First, EDP-aware planning yields large gains under contention***: For $N{=}7$ and $K{=}50$, RCORE reduced EDP by 91.22% compared to BL on average, while simultaneously improving energy (69.5% reduction) and execution time (71.9% reduction). This shows that our technique does not improve one metric at the expense of the other, but simultaneously reduces energy consumption and improves execution time. ***Secondly, AMP-awareness matters.*** The homogenized version achieved similar energy but was much slower, doubling EDP relative to RCORE. This highlights that, on AMPs, the choice of P-/E-core allocations is as important as the decision to run containers concurrently. ***Finally, the compared strategies generalize poorly to AMP.*** Their fixed policies do not account for heterogeneous core capabilities or the variability of parallel applications under oversubscription, leading to systematically suboptimal placements. While RR was the strongest non-RCORE competitor, it still resulted in 38% higher EDP than RCORE. SJF and pinned serial variants achieved partial improvements over BL, yet were several times worse in all metrics of comparison to RCORE.

VI. Conclusion

This paper presented RUNCORE, an energy-aware resource manager for containerized workloads on AMPs. By combining offline EDP profiling with a three-phase runtime allocation pipeline, it efficiently maps Dockerized applications onto P- and E-cores. Experiments on an Intel Core i9-12900K under heavy oversubscription showed that RUNCORE reduced mean batch-wide EDP by up to 91.2% relative to the serial baseline and consistently outperformed classical container scheduling strategies, achieving improvements of up to 85.54%. These results demonstrate that asymmetry-aware resource management is a key enabler of energy-efficient container execution on next-generation cloud and edge platforms. Future work includes replacing the offline stage with a lightweight strategy, as well as integrating RUNCORE with Kubernetes/Apptainer for multi-node asymmetric-core resource management.

References

[1] J. C. Sáez, A. Pousa, R. Rodríguez-Rodríguez, F. Castro, and M. Prieto-Matias, "An evaluation of performance-asymmetric multicore processors with heterogeneous task parallelism," *Concurr. Comput. Pract. Exp.*, vol. 34, no. 18, p. e7052, 2022.

[2] ARM Ltd., "big.LITTLE technology: The future of mobile," White Paper, 2013. [Online]. Available: https://developer.arm.com/documentation

[3] R. Schöne, T. Ilsche, M. Bielert, A. Gocht-Zech, and D. Hackenberg, "Energy efficiency features of the Intel Alder Lake architecture," in *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2024, pp. 146–155.

[4] R. Kumar, D. M. Tullsen, P. Ranganathan, N. P. Jouppi, and K. I. Farkas, "Single-ISA heterogeneous multi-core architectures for multithreaded workload performance," in *Proceedings of the 31st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2004, pp. 64–75.

[5] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux J.*, vol. 2014, no. 239, p. 2, 2014.

[6] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, omega, and Kubernetes," *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016.

[7] P. Liu and J. Guitart, "Performance characterization of containerization for hpc workloads on infiniband clusters: an empirical study," *Cluster Computing*, vol. 25, no. 2, pp. 847–868, 2022.

[8] "Amazon ec2 mac instances," https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-mac-instances.html, 2024, accessed: 2026-02-12.

[9] Y. Kwon, C. Kim, S. Maeng, and J. Huh, "Virtualizing performance asymmetric multi-core systems," in *Proceedings of the 38th Annual International Symposium on Computer Architecture (ISCA)*. New York: ACM, 2011, pp. 45–56.

[10] G. Rattihalli, M. Govindaraju, H. Lu, and D. Tiwari, "Exploring potential for non-disruptive vertical auto scaling and resource estimation in Kubernetes," in *Proceedings of the IEEE International Conference on Cloud Computing (CLOUD)*. IEEE, 2019, pp. 33–40.

[11] D. Balla, C. Simon, and M. Maliosz, "Adaptive scaling of Kubernetes pods," in *Proceedings of the IEEE/IFIP Network Operations and Management Symposium (NOMS)*. IEEE, 2020, pp. 1–5.

[12] J. C. Saez and M. Prieto-Matias, "Evaluation of the intel thread director technology on an alder lake processor," in *ACM SIGOPS APSYS*, 2022, pp. 61–67.

[13] C. Bilbao, J. C. Saez, and M. Prieto-Matias, "Flexible system software scheduling for asymmetric multicore systems with pmcsched: A case for intel alder lake," *CCPE*, p. e7814, 2023.

[14] V. Dalakoti and D. Chakraborty, "Apple m1 chip vs intel (x86)," *IJRD*, vol. 7, no. 5, pp. 207–211, 2022.

[15] B. Salami, H. Noori, and M. Naghibzadeh, "Online energy-efficient fair scheduling for heterogeneous multi-cores considering shared resource contention," *The Journal of Supercomputing*, pp. 1–20, 2022.

[16] M. Shimchenko, E. Österlund, and T. Wrigstad, "Scheduling garbage collection for energy efficiency on asymmetric multicore processors," *arXiv preprint arXiv:2403.02200*, 2024.

[17] R. Schöne, M. Velten, D. Hackenberg, and T. Ilsche, "Energy efficiency features of the intel alder lake architecture," in *ICPE*, 2024, pp. 95–106.

[18] A. F. Lorenzon, C. C. De Oliveira, J. D. Souza, and A. C. S. Beck, "Aurora: Seamless optimization of openmp applications," *TPDS*, vol. 30, no. 5, pp. 1007–1021, 2018.

[19] M. A. Suleman, M. K. Qureshi, and Y. N. Patt, "Feedback-driven threading: power-efficient and high-performance execution of multi-threaded workloads on cmps," *ACM Sigplan Notices*, vol. 43, no. 3, pp. 277–286, 2008.

[20] A. Iosup, S. Ostermann, M. N. Yigitbasi, R. Prodan, T. Fahringer, and D. Epema, "Performance analysis of cloud computing services for many-tasks scientific computing," *IEEE Transactions on Parallel and Distributed systems*, vol. 22, no. 6, pp. 931–945, 2011.

[21] J. Higgins, V. Holmes, and C. Venters, "Orchestrating docker containers in the hpc environment," in *International Conference on High Performance Computing*. Springer, 2015, pp. 506–513.

[22] R. R. Expósito, G. L. Taboada, S. Ramos, J. Touriño, and R. Doallo, "Performance analysis of hpc applications in the cloud," *Future Generation Computer Systems*, vol. 29, no. 1, pp. 218–229, 2013.

[23] H. Zhang and H. Hoffmann, "Maximizing performance under a power cap: A comparison of hardware, software, and hybrid techniques," *ACM SIGPLAN Notices*, vol. 51, no. 4, pp. 545–559, 2016.

[24] M. Etinski, J. Corbalan, J. Labarta, and M. Valero, "Optimizing job performance under a given power constraint in hpc centers," in *International Conference on Green Computing*. IEEE, 2010, pp. 257–267.

[25] V. S. da Silva, E. C. de Lima, J. Schwarzrock, F. D. Rossi, M. C. Luizelli, A. C. S. Beck, and A. F. Lorenzon, "Synergistically rebalancing the edp of container-based parallel applications," *IEEE Transactions on Parallel and Distributed Systems*, vol. 35, no. 3, pp. 484–498, 2024.

[26] Y. Kwon, C. Kim, S. Maeng, and J. Huh, "Virtualizing performance asymmetric multi-core systems," *ACM SIGARCH Computer Architecture News*, vol. 39, no. 3, pp. 45–56, 2011.

[27] B. Teabe, P. L. Wapet, A. Tchana, and D. Hagimont, "Dealing with performance unpredictability in an asymmetric multicore processor cloud," in *European Conference on Parallel Processing*. Springer, 2017, pp. 332–344.

[28] D. H. Bailey, E. Barszcz, J. T. Barton *et al.*, "The NAS parallel benchmarks," *Int. J. High Perform. Comput. Appl.*, vol. 5, no. 3, pp. 63–73, 1991.

[29] R. Gonzalez and M. Horowitz, "Energy dissipation in general purpose microprocessors," *IEEE J. Solid-State Circuits*, vol. 31, no. 9, pp. 1277–1284, 1996.